Université de Caen Normandie

IUT Grand Ouest Normandie

Bachelor Universitaire de Technologie

Science des Données

Campus de Lisieux

Science des Données 1 - SAE 1.01

Reporting et exploration à partir d'un SGBD

Thématique

Analyse des universités mondiales

Auteurs

BAMBELA MOUANDA Yanh
JUMEL Martin
MBANG NDONG Glaineta

Année universitaire 2025-2026

Table des matières

Introduction **3**

1 Les missions confiées à Yanh **4**
1.1 Présentation des fonctions 4
1.2 Difficultés rencontrées 4
1.3 Solutions mises en œuvre 5
1.4 Compétences développées et pistes d'amélioration 5

2 Les missions confiées à Glaineta **6**
2.1 top5_pays_par_critere 6
2.2 repartition_universites_par_region 6
2.3 n_universites_par_indicateur 7
2.4 stats_migration_vs_perf 7
2.5 Compétences développées et pistes d'améliorations 8

3 Les missions confiées à Martin **9**
3.1 kpis_dashboard 9
3.2 rechercher_universites 9
3.3 top20_universites 10
3.4 stats_pib_vs_perf 10
3.5 stats_mixite_vs_perf 11
3.6 Compétences développées et pistes d'améliorations 12

Table des figures **13**

Introduction

Dans cette SAE 1.01, nous travaillons sur une base de données relationnelle couvrant la période 2016–2025, regroupant à la fois les classements des universités et des indicateurs socio-économiques des pays (PIB, alphabétisation, population, égalité femmes-hommes, etc.) à travers plusieurs tables (régions, classements, pays et universités). L'objectif est d'explorer les liens entre le développement socio-économique et la performance académique, en analysant l'influence de différents facteurs sur la qualité des universités.

La SAE se concentre sur le reporting et l'exploration de la base de données relationnelle présentée brièvement ci-dessus. Un total de 13 fonctions appartenant chacune à l'un de ces deux grands axes ont été réparties entre nous trois : Yanh, Glaine et Martin. L'objectif est d'expliquer ce que représente chaque fonction, quelle a été notre démarche algorithmique pour pouvoir remplir les rôles associés à chaque fonction et les difficultés que nous avons chacun rencontrées lors de l'écriture de ces fonctions. Chaque partie représente le travail fait par chacun d'entre nous durant cette SAE.

Tout d'abord, avant de rentrer au cœur de chacune des parties de ce rapport, il nous faut présenter plus en profondeur la base de données composée des quatre tables citées plus haut.

Voici la structure des tables de la base de données :

- **regions**(id_reg, nom_reg)
- **pays**(id_pays, nom_pays, #id_reg, population, pib_par_hab, superf_m2, migration_nette, part_industrie, part_services, taux_alphabet, tel_pour_1000_hab)
- **universites**(id_univ, nom_univ, #id_pays)
- **classements**(id_class, #id_univ, annee, rang, score_the, score_enseig, score_env_rech, score_qualite_rech, score_rel_internat, score_impact_indust, ratio_hom, ratio_fem, ratio_etud_pers, population_etud, pct_etud_internat)

Intéressons-nous également à leur relation entre-elles :

relation entre regions et pays : Un-à-Plusieurs (1 :N).

- **Justification :** Une région (ex : Europe) peut regrouper plusieurs pays, mais un pays n'appartient qu'à une seule région dans ce modèle. La clé étrangère `#id_reg` dans la table `pays` permet de lier chaque nation à sa zone géographique sans avoir à répéter le nom de la région pour chaque ligne.

relation entre pays et universites : Un-à-Plusieurs (1 :N).

- **Justification :** Un pays peut abriter de nombreuses universités, tandis qu'une université est géographiquement rattachée à un seul pays. L'utilisation de `#id_pays` dans la table `universites` permet d'accéder aux données macro-économiques du pays (PIB, population) pour chaque établissement par une simple jointure.

relation entre universites et classements : Un-à-Plusieurs (1 :N).

- **Justification :** Une université est évaluée sur plusieurs années. Il y a donc une seule entrée pour l'identité de l'université (`universites`), mais plusieurs entrées de scores (`classements`) qui lui sont liées via `#id_univ`. Cela permet de conserver l'historique des performances sans dupliquer inutilement le nom de l'université à chaque nouvelle année de classement.

Enfin, il y a 2 fonctions communes que nous devons introduire : ouvrir_connexion_bdd() et fermer_connexion_bdd(). La première permet d'établir la connexion à notre base de données SQLite et paramètre les résultats sous forme de dictionnaire, pour nous aider à l'analyse. La seconde permet donc de fermer la connexion à la base de données.

Pour construire la première fonction, on regarde d'abord si le chemin vers la base de données existe, dans le cas contraire, on affiche un message d'erreur et on renvoie None. Sinon, on continue en établissant la connexion à la base de données grâce à sqlite3.connect() et on paramètre les résultats sous forme d'un dictionnaire avec connexion.row_factory = sqlite3.Row. Finalement, on renvoie la connexion et on affiche un message montrant le succès de l'opération.

La deuxième fonction se fait plus facilement : on regarde seulement s'il y a une connexion, auquel cas, on ferme la connexion et on affiche un message montrant le succès de l'opération. Sinon, on affiche un message d'erreur.

L'écriture de ces deux fonctions n'a soulevé aucune difficulté.

```python
if not os.path.exists(chemin_vers_bdd):
    print("Le fichier de la base de données n'existe pas.")
    return None
else:
    connexion = sqlite3.connect(chemin_vers_bdd)
    connexion.row_factory = sqlite3.Row
    print("Connexion à la base de données établie.")
    return connexion
```

```python
if connexion:
    connexion.close()
    print("Connexion à la base de données fermée.")
else:
    print("Aucune connexion active à fermer.")
return None
```

Fig. 1. ouvrir_connexion_bdd() (à gauche) et fermer_connexion_bdd() (à droite)

1 Les missions confiées à Yanh

Dans le cadre de la SAE 1.01, j'ai participé au développement et à l'implémentation de plusieurs fonctions du programme Python. Mon travail a surtout porté sur le calcul de statistiques. Les fonctions que j'ai réalisées ou complétées sont les suivantes :

- `top10_pays_multi_annees`
- `stats_internationalisation_vs_recherche`
- `evolution_universite`
- `repartition_par_alphabetisation`

1.1 Présentation des fonctions

— **top10_pays_multi_annees** : permet de calculer le top 10 des pays sur plusieurs années (de 2020 à 2025) en utilisant une pondération pour tenir compte du nombre d'universités par pays.
— **stats_internationalisation_vs_recherche** : analyse le lien entre le pourcentage d'étudiants internationaux et les performances en recherche et le score THE.
— **evolution_universite** : permet de suivre l'évolution des scores d'une université donnée au fil des années
— **repartition_par_alphabetisation** : calcule la répartition des universités en fonction du taux d'alphabétisation du pays auquel elles appartiennent.

```python
if not nom_univ:
    return []
cur = connexion.cursor()

# On sélectionne les scores de l'université année par année
# en filtrant sur le nom (LIKE permet une recherche partielle)
req = """
SELECT c.annee, c.score_enseig, c.score_qualite_rech, c.score_the
FROM classements c
JOIN universites u ON c.id_univ = u.id_univ
WHERE u.nom_univ LIKE ?
ORDER BY c.annee
"""

# On retourne la liste des résultats sous forme de tuples
return [tuple(ligne) for ligne in cur.execute(req, ('%' + nom_univ + '%',))]
```

Fig. 2. Code de la fonction evolution_universite

1.2 Difficultés rencontrées

La principale difficulté a été de bien comprendre la structure de la base de données, en particulier les relations entre les tables "classements", "universites", "pays" et "regions". Il fallait être sûr d'utiliser les bons identifiants lors des jointures pour éviter des erreurs de résultats ou des doublons.

Une autre difficulté a été le choix des classes pour certaines analyses, par exemple pour l'internationalisation des étudiants ou le taux d'alphabétisation. Il fallait trouver des intervalles qui soient à la fois simples et pertinents pour l'analyse.

L'une des difficultés a été de bien distinguer les niveaux pays et région. Au début, je mélangeais certaines valeurs, ce qui donnait des résultats incorrects. Pour résoudre ce problème, j'ai stocké temporairement les valeurs de PIB et d'alphabétisation dans des listes intermédiaires, puis j'ai calculé les moyennes uniquement à la fin.

Pour la fonction top10_pays_multi_annees, le plus compliqué a été de trouver une méthode pour ne pas avantager les pays qui ont très peu d'universités, tout en gardant une comparaison juste.

```python
passcur = connexion.cursor()

# On calcule le score moyen du critère pour chaque pays
# ainsi que le nombre d'universités sur la période
req = f"""
SELECT p.nom_pays, AVG(c.{critere}) AS moy, COUNT(DISTINCT u.id_univ) AS nb
FROM classements c
JOIN universites u ON c.id_univ = u.id_univ
JOIN pays p ON u.id_pays = p.id_pays
WHERE c.annee BETWEEN ? AND ?
GROUP BY p.nom_pays
HAVING nb >= 20
"""

res = []

# On applique une pondération pour éviter qu'un pays avec peu
# d'universités soit trop favorisé
for r in passcur.execute(req, (annee_deb, annee_fin)):
    score = r["moy"] * r["nb"] / (r["nb"] + 20)
    res.append((r["nom_pays"], round(score, 3), r["nb"]))

# On trie les pays selon le score pondéré et on garde les 10 premiers
return sorted(res, key=lambda x: x[1], reverse=True)[:10]
```

Fig. 3. Code de la fonction top10_pays_multi_annees

1.3 Solutions mises en œuvre

Pour les jointures, j'ai utilisé systématiquement les clés primaires et étrangères (‘id_univ‘, ‘id_pays‘, ‘id_reg‘) afin d'assurer la cohérence des données entre les tables.

Pour les classes (internationalisation et alphabétisation), j'ai utilisé des structures ‘CASE WHEN‘ en SQL afin de regrouper les valeurs en catégories faciles à interpréter.

Pour le top 10 des pays, j'ai mis en place une formule de pondération qui combine le score moyen et le nombre d'universités. Cela permet d'éviter qu'un pays avec très peu d'universités mais un score élevé soit surévalué par rapport à un pays plus représentatif.

1.4 Compétences développées et pistes d'amélioration

Cette SAE m'a permis de mieux comprendre comment utiliser des données réelles pour faire une analyse concrète. Avant ce projet, je savais écrire des requêtes SQL, mais j'avais surtout l'habitude de faire des exercices isolés. Ici, j'ai appris à relier plusieurs tables, à choisir les bons indicateurs et à produire des résultats qui ont du sens. J'ai aussi progressé sur l'utilisation de Python pour automatiser le traitement des données. Le fait de passer par un script pour interroger la base et générer un fichier Excel m'a montré comment on peut créer un véritable outil d'analyse, et pas seulement faire des calculs. Cette SAE m'a également appris à être plus critique sur les résultats. Par exemple, avec le top 10 des pays, j'ai compris qu'un bon score moyen ne suffit pas si le pays a très peu d'universités, d'où l'intérêt d'utiliser une pondération. De la même manière, pour les graphiques, j'ai appris à réfléchir à la lisibilité et à la pertinence du type de graphique.

Plusieurs améliorations pourraient être envisagées pour la suite du projet. Par exemple, on pourrait ajouter des graphiques d'évolution dans le temps pour visualiser plus clairement la progression ou la baisse des universités ou des pays. Il serait aussi intéressant d'ajouter d'autres indicateurs, comme le budget consacré à l'éducation ou le nombre de chercheurs, afin d'enrichir l'analyse. Enfin, on pourrait rendre l'outil plus interactif, par exemple en permettant à l'utilisateur de choisir une université ou un pays directement depuis l'interface Excel ou via un petit formulaire.

2 Les missions confiées à Glaineta

Dans le cadre de la SAE, j'ai écrit plusieurs fonctions variées dans leur demande. J'ai dû par ailleurs tracer plusieurs graphes associés aux fonctions. Je présente la démarche algorithmique adoptée, les difficultés rencontrées ainsi que les solutions mises en place pour chaque fonction. Cette partie repose sur l'implémentation des fonctions suivantes :

- `top5_pays_par_critere`
- `repartition_universites_par_region`
- `n_universites_par_indicateur`
- `stats_migration_vs_perf`

2.1 top5_pays_par_critere

Cette fonction permet de identifier les 5 pays dont les universités présentent la meilleure performance moyenne pour un critère donné (enseignement ou recherche) sur une année donnée.

Dans un premier temps, les données nécessaires sont récupérées via un curseur, qui sert de zone tampon entre Python et la base de données SQLite. Ensuite, une requête SQL est construite afin d'identifier le top 5 des pays dont les universités présentent la meilleure performance moyenne pour un critère donné (enseignement ou recherche). La nature du critère est vérifiée à l'aide d'une condition (if) ; si celui-ci n'est pas valide, la fonction renvoie une liste vide. Pour calculer les performances moyennes, une fonction d'agrégation (AVG) est appliquée aux colonnes score_enseig ou score_qualite_rech de la table classements. Comme l'analyse porte sur une année précise, une clause WHERE est utilisée pour filtrer les données. Enfin, les clauses ORDER BY et LIMIT permettent de trier les résultats par moyenne décroissante et de ne conserver que les cinq premiers pays. Du côté de Python, la méthode fetchall() est utilisée pour récupérer les résultats de la requête SQL et les retourner sous forme d'une liste de tuples. Concernant l'application Excel, une feuille nommée Top5 Pays–Enseignement (2025) a été créée à l'aide de l'objet Python de la bibliothèque openpyxl (ws.nom_de_la_fonction()). Les en-têtes ont été ajoutés grâce à la méthode append(), puis une boucle for a permis d'insérer ligne par ligne les résultats renvoyés par notre fonction. Enfin, l'ajout d'un diagramme à barres a été réalisé en utilisant la fonction generer_bar_charts précédemment créée. La même procédure a été appliquée pour les critères score_qualite_rech et score_the.

Lors du développement de cette fonction, le principal problème a été l'utilisation de la fonction d'agrégation AVG sans clause GROUP BY, ce qui entraînait le calcul d'une moyenne globale et le retour d'une seule ligne, choisie arbitrairement pour un pays. L'ajout de GROUP BY a résolu ce problème. Par ailleurs, il fallait s'assurer que le critère soit limité à score_enseig, score_qualite_rech ou score_THE. Cet oubli n'a pas causé d'erreur dans le code, mais ne respectait pas les consignes.

```
if critere not in ("score_enseig", "score_qualite_rech", "score_the"):
    return []

curs = connexion.cursor()
req = """
    SELECT p.nom_pays, ROUND(AVG(c.score_enseig), 3) as score_moyen FROM pays AS p
    JOIN universites AS u ON p.id_pays = u.id_pays
    JOIN classements AS c ON c.id_univ = u.id_univ
    WHERE c.annee = ?
    GROUP BY p.nom_pays
    ORDER BY score_moyen DESC
    LIMIT 5 ;
    """
curs.execute(req, (annee,))
return [tuple(ligne) for ligne in curs.fetchall()]
```

Fig. 4. Code de la fonction top5_pays_par_critere

2.2 repartition_universites_par_region

Cette fonction permet de calculer, pour une année donnée, le nombre d'universités par région et retourner ces informations sous forme de liste de tuples.

Dans cette fonction, comme pour la précédente, un curseur est utilisé pour récupérer les données. Une requête SQL est ensuite construite pour calculer la répartition des universités par région pour une année donnée. Pour cela, plusieurs tables sont reliées via des JOIN...ON (pays, classements et regions) et la clause WHERE filtre l'année souhaitée. Enfin, les résultats sont récupérés en Python sous forme de liste de tuples grâce à fetchall() sur le curseur. Dans l'application Excel, une feuille nommée Univ par Région a été créée via l'objet "ws" Python de la bibliothèque openpyxl. Après avoir ajouté les en-têtes avec append(), une boucle for a permis d'insérer les résultats ligne par ligne, puis un diagramme circulaire a été généré à l'aide de la fonction generer_pie_charts déjà définie.

Pour cette fonction, la difficulté n'a pas été dans l'écriture du code, mais dans la compréhension de la consigne. En effet, la trace d'exécution et le docstring indiquaient que la fonction devait renvoyer une liste, alors que la consigne précisait un dictionnaire. Il a donc été nécessaire de solliciter l'avis de la professeure et des collègues pour clarifier ce quiproquo.

```python
curs = connexion.cursor()
req = """
    SELECT r.nom_reg, COUNT(DISTINCT u.nom_univ) AS nbre_univ
    FROM universites AS u
    JOIN pays AS p ON u.id_pays = p.id_pays
    JOIN regions AS r ON r.id_reg = p.id_reg
    JOIN classements  AS c ON c.id_univ = u.id_univ
    WHERE c.annee = ?
    GROUP BY r.nom_reg;
"""
curs.execute(req, (annee,))
return [tuple(ligne) for ligne in curs.fetchall()]
```

Fig. 5. Code de la fonction repartition_universites_par_region

2.3 n_universites_par_indicateur

Cette fonction classe les universités pour une année donnée selon un indicateur de performance choisi, en renvoyant soit le Top N, soit le Bottom N selon le paramètre sens, en ignorant les valeurs NULL.

La création de cette fonction commence par la vérification de la validité de l'indicateur de performance, qui doit être score_enseig ou score_qualite_rech ; sinon, la fonction renvoie une liste vide. Selon le paramètre sens, un ordre croissant ou décroissant est déterminé via un test if. La requête SQL sollicite les tables pays, universites et classements, reliées par des JOIN ... ON. La clause ORDER BY permet de trier les résultats selon l'indicateur et LIMIT restreint le nombre de lignes renvoyées au Top ou Bottom N. Enfin, les résultats sont récupérés avec fetchall() et retournés sous forme de liste de tuples. Au sein de l'application Excel, une feuille nommée Top 5 Univ – Enseignement a été créée à l'aide de la bibliothèque Python openpyxl notamment avec "ws". Après l'ajout des en-têtes via append(), les données ont été insérées ligne par ligne à l'aide d'une boucle for.

Au départ, le problème venait du fait que la requête SQL n'était pas correctement intégrée au code Python. Les variables indicateur et order n'étaient donc pas interprétées par SQL, ce qui provoquait une erreur d'exécution. Par ailleurs, la fonction devait renvoyer une liste de tuples ; or, dans la première version, elle retournait directement le résultat de curs.fetchall(). L'instruction [tuple(ligne) for ligne in curs.fetchall()] a ainsi été ajoutée afin de garantir le format attendu.

```python
# Vérification que l'indicateur est valide
if indicateur not in ("score_enseig", "score_qualite_rech"):
    return []

# Détermination de l'ordre
if sens == "top":
    order = "DESC"
else:
    order = "ASC"

# Construire la requête SQL
req = ("SELECT c." + indicateur + " AS valeur, u.nom_univ, p.nom_pays, c.rang "
    "FROM pays AS p "
    "JOIN universites AS u ON p.id_pays = u.id_pays "
    "JOIN classements AS c ON c.id_univ = u.id_univ "
    "WHERE c.annee = ? "
    "ORDER BY c." + indicateur + " " + order + " "
    "LIMIT ?;")

curs = connexion.cursor()
curs.execute(req, (annee, n))
return [tuple(ligne) for ligne in curs.fetchall()]
```

Fig. 6. Code de la fonction n_universites_par_indicateur

2.4 stats_migration_vs_perf

Cette fonction répartit les pays selon la valeur de leur migration nette (négative, nulle ou positive), puis calcule pour chaque catégorie la moyenne des scores d'enseignement et de qualité de la recherche des universités correspondantes.

Après la création du curseur, le code se poursuit avec une requête SQL visant à classer les pays selon leur migration nette. Celle-ci pouvant être négative, nulle ou positive, l'instruction CASE ... END est utilisée afin de gérer ces trois situations. Pour chaque catégorie, la fonction d'agrégation AVG() est appliquée aux colonnes score_enseig et score_qualite_rech, avec un arrondi à trois décimales grâce à la fonction ROUND(). Les tables pays, universites et classements sont reliées à l'aide de JOIN ... ON afin d'associer les pays, leur migration nette et les scores moyens des universités correspondantes. Côté Python, les résultats de la requête sont récupérés avec fetchall(). Un dictionnaire vide resultat est ensuite créé et alimenté à l'aide d'une boucle for, afin de stocker les moyennes par classe de migration sous forme de dictionnaires imbriqués. La fonction retourne finalement ce dictionnaire. Dans l'application Excel, une feuille nommée Migration vs Perf est créée à l'aide de la bibliothèque Python openpyxl. Les en-têtes sont ajoutés avec la méthode append(), puis les données sont insérées ligne par ligne dans la feuille à l'aide d'une boucle for.

Lors du développement de cette fonction, plusieurs difficultés ont été rencontrées. Tout d'abord, la colonne migration_nette pouvant prendre des valeurs négatives, nulles ou positives, il a été nécessaire de gérer ces trois cas distincts, ce qui a conduit à l'utilisation de l'instruction CASE ... END. Ensuite, une erreur est survenue lors du regroupement des données : le GROUP BY avait initialement été appliqué aux colonnes score_enseig et score_qualite_rech, alors que celles-ci faisaient déjà l'objet d'une fonction d'agrégation (AVG), ce qui a provoqué une incohérence SQL. Par

ailleurs, la principale difficulté a été de déterminer la méthode permettant d'avoir des dictionnaires imbriqués dans un dictionnaire d'où la création d'un dictionnaire resultat et l'utilisation d'une boucle for qui permettra de stocker les données, déjà dans des dictionnaires, dans ce dictionnaire.

```
curs = connexion.cursor()
req="""
    SELECT ROUND(AVG(score_enseig),3) AS moy_enseig, ROUND(AVG(score_qualite_rech),3) AS moy_rech,
        CASE
            WHEN p.migration_nette < 0 THEN 'Négative'
            WHEN p.migration_nette = 0 THEN 'Nulle'
            ELSE 'Positive'
        END AS classe_migration
    FROM  pays AS p
    JOIN universites AS u ON u.id_pays = p.id_pays
    JOIN classements AS c ON u.id_univ = c.id_univ
    WHERE annee = ?
    GROUP BY classe_migration;
    """
curs.execute(req, (annee,))
zone_stockage = curs.fetchall()

resultat = {}

for moy_enseig, moy_rech, classe_migration in zone_stockage:
    resultat[classe_migration] = {
    "moy_enseig": moy_enseig,
    "moy_rech": moy_rech
}
return resultat
```

Fig. 7. Code de la fonction stats_migration_vs_perf

2.5 Compétences développées et pistes d'améliorations

Cette partie du projet m'a permis de renforcer mes compétences en conception algorithmique et en débogage. J'ai également amélioré ma capacité à structurer du code et des requêtes SQL clairs et fonctionnels. Plus précisément, j'ai approfondi ma compréhension de l'utilisation de fetchall(), de l'instruction CASE et des requêtes paramétrées (?). Par ailleurs, ce travail m'a aidé à mieux distinguer l'usage des listes et des dictionnaires.

Enfin, des améliorations possibles incluent la mise place des modèles simples de prévision (régression linéaire, moyenne mobile) pour estimer l'évolution future d'indicateurs comme le budget ou le nombre de chercheurs. Il est intéressant d'étudier les variations des indicateurs sur plusieurs années pour identifier des tendances à long terme et repérer d'éventuelles anomalies. Il serait pertinent de renforcer l'aspect visuel des graphiques en adaptant les couleurs et les échelles, en combinant plusieurs types de représentations et en ajoutant des annotations pour faciliter l'interprétation.

3 Les missions confiées à Martin

Dans le cadre de cette SAE, mon rôle était certes celui de codeur, et j'ai également assisté Glaineta dans l'importation de ses fonctions sur Excel. 5 fonctions m'ont été attribuées :

- `kpis_dashboard`
- `rechercher_universites`
- `top20_universites`
- `stats_pib_vs_perf`
- `stats_mixite_vs_perf`

3.1 kpis_dashboard

Pour cette fonction, l'objectif est de calculer les indicateurs globaux à partir de la base de données. Les indicateurs demandés sont entre autres : le nombre d'universités, de pays, de régions, des moyennes comme celui du PIB par habitant etc.

Pour l'écriture de cette fonction, le traitement peut se faire directement par la requête SQL. Dans un premier temps, j'ai pensé à faire plusieurs jointures : entre la table pays, universites, regions et classements. Cependant, cela a posé problème car les jointures entre plusieurs tables peuvent multiplier les lignes lorsque plusieurs correspondances existent, ce qui risque de fausser le calcul de certaines moyennes ou agrégats, ce qu'on calcule ici. Et cela s'est traduit par une différence entre les valeurs trouvées et les valeurs trouvées par l'énoncé. Pour cette raison, les indicateurs globaux ont été calculés par des requêtes séparées sur chaque table, évitant ainsi toute duplication et garantissant des valeurs correctes. Ensuite, nous exécutons la requête et retournons sous forme de dictionnaire le résultat avec dict(curseur.fetchone()). Au départ, je voulais utiliser un .fetchall() mais cela renvoyait une erreur car fetchone() renvoie une seule ligne convertible en dictionnaire, alors que fetchall() renvoie une liste de lignes, que dict() ne peut pas convertir directement. On peut convertir fetchall() en une liste de dictionnaires en traitant chaque ligne, mais je me suis rendu compte que le résultat ne représentait qu'une seule ligne, donc cela serait trop long et inutile.

In fine, la seule réelle difficulté dans l'écriture de cette fonction est de comprendre précisément pourquoi une jointure pose problème.

Enfin, il fallait retranscrire dans Excel cette fonction. Au départ, j'avais beaucoup de mal car je ne savais pas du tout comment faire. En suivant le TP de préparation, cela s'est fait beaucoup plus naturellement et j'ai pu comprendre la logique et les différentes étapes à suivre pour créer une feuille Excel.

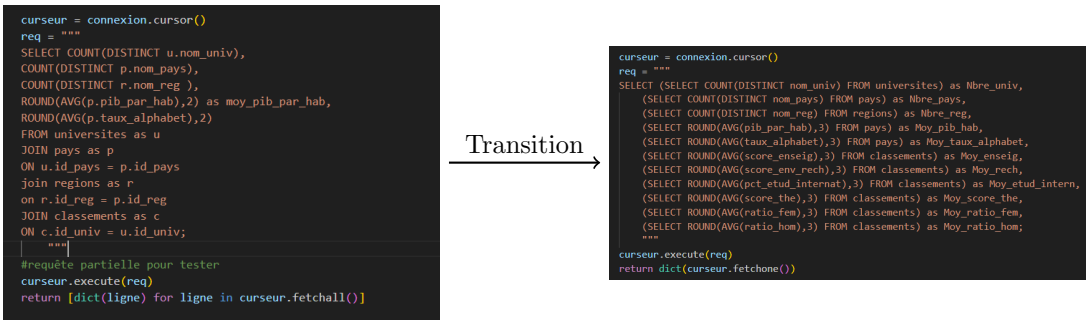

Fig. 8. Schéma de transition : de la première version à la dernière version de kpis_dashboard.

3.2 rechercher_universites

Cette fonction a pour objectif de rechercher des universités selon des critères optionnels et les range selon leur rang.

Une partie de cette fonction doit être traitée sur Python car le caractère optionnel des critères rend le traitement impossible sur la requête SQL directement. Cela a été la plus grosse difficulté lors de l'écriture de la fonction : comment traduire ce caractère "optionnel".

Pour ce faire, il fallait donc d'abord seulement importer ce qui était demandé avec la requête SQL : le rang, le nom de l'université, le nom du pays etc. en faisant les jointures pour mener à bien l'importation. Ensuite, j'ai décidé de créer deux listes qui serviront à l'une de stocker les conditions et à l'autre les paramètres pour l'exécution de la requête. L'étape d'après était de vérifier si chaque paramètre de la fonction était égale à None ou non. Dans le cas contraire, elle ajoute dynamiquement les critères de filtrage optionnels (pays, année, scores minimaux etc.) dans la liste des critères et elle ajoute dans la liste des paramètres le paramètre rentré (par exemple "France" pour nom_pays). Après avoir eu cette idée, j'ai pas mal bloqué sur la manière de l'ajouter à la requête : je savais qu'il fallait ajouter WHERE et AND mais je ne savais pas comment car il fallait mettre WHERE, le critère et AND et j'étais bloqué car une boucle était impossible à faire (il fallait totalement modifier la structure de mon code et SQL n'accepte pas plusieurs WHERE).

J'ai donc relu le TP sur les chaînes de caractères pour voir s'il y existait une solution à ce problème et le .join() l'est : il va prendre tous les éléments de la liste de conditions et les concaténer en mettant " AND " entre chaque élément, sans que le Where soit dupliqué (car il n'y a pas de boucle). Enfin, j'ai pensé au dernier moment à rendre la fonction plus robuste en ajoutant des vérifications de type et de validité des paramètres (en début de fonction). Cela permet de s'assurer que les arguments fournis sont corrects. Cette vérification prévient les erreurs d'exécution.

À la fin, on exécute avec notre liste de paramètres et on renvoie une liste de tuples, comme demandé. L'importation sur Excel s'est faite sans problème : on crée la feuille, on ajoute les en-têtes et on ajoute le contenu de la fonction paramétrée.

Finalement, les 2 réelles difficultés de cette fonction ont été la manière de rendre une requête SQL dynamique et savoir concaténer des chaînes de caractères avec les bons outils.

```python
if pays is not None and not isinstance(pays, str):
    raise ValueError("pays doit être une chaîne de caractères")
if annee is not None and not isinstance(annee, int):
    raise ValueError("annee doit être un entier")
if min_enseig is not None and not isinstance(min_enseig, (int, float)):
    raise ValueError("min_enseig doit être un nombre")
if min_rech is not None and not isinstance(min_rech, (int, float)):
    raise ValueError("min_rech doit être un nombre")
if not isinstance(limit, int) or limit <= 0:
    raise ValueError("limit doit être un entier positif")

curseur = connexion.cursor()
req = """
SELECT c.rang, u.nom_univ, p.nom_pays, c.score_enseig, c.score_qualite_rech, c.score_the
FROM classements AS c
JOIN universites AS u ON u.id_univ = c.id_univ
JOIN pays AS p ON p.id_pays = u.id_pays
"""

conditions = []
parametres = []

# On vérifie si chaque paramètre est égal à None
if pays is not None:
    conditions.append("p.nom_pays = ?")
    parametres.append(pays)

if annee is not None:
    conditions.append("c.annee = ?")
    parametres.append(annee)

if min_enseig is not None:
    conditions.append("c.score_enseig >= ?")
    parametres.append(min_enseig)

if min_rech is not None:
    conditions.append("c.score_qualite_rech >= ?")
    parametres.append(min_rech)

if conditions:
    req += " WHERE " + " AND ".join(conditions)

req += " ORDER BY c.rang ASC LIMIT ?"
parametres.append(limit)

curseur.execute(req, parametres)
resultats = curseur.fetchall()
return [tuple(ligne) for ligne in resultats]
```

	A	B	C	D	E	F
1	Rang	Université	Pays	Score Enseig...	Score Recher...	Score THE
2	42	Paris Science...	France	74.40	82	77.67
3	64	Université Pa...	France	68.40	81.50	73.48
4	71	Institut Polyt...	France	69.20	73.40	71.27
5	76	Sorbonne Un...	France	64.60	82.40	70.42
6	184	Université Pa...	France	44.70	90.70	59.57
7	287	École Normal...	France	56.70	53	54.28
8	365	Université Gr...	France	41.70	64.50	50.63
9	375	Institut Agro	France	47.70	60.70	50.22
10	380	IMT Atlantique	France	47.40	51.30	50.04
11	408	Aix-Marseille ...	France	43.50	66.70	49.11
12	519	Centrale Nan...	France	41	47.10	45.24
13	550	ENSTA Breta...	France	40.20	52.90	44.48
14	572	École Nation...	France	41.50	48.60	44.10
15	731	École des Mi...	France	42.60	32.50	39.76
16	934	National Vete...	France	44.30	44.30	35.81

Fig. 9. Code de la fonction rechercher_universites (à gauche) et resultat sur Excel en prenant comme paramètre nom_pays = "France", annee = "2025" et min_enseig = "40.0" (à droite)

3.3 top20_universites

Cette fonction renvoie les 20 meilleures universités selon le rang pour une année donnée.

Le traitement de la fonction peut se faire sur SQL. La requête doit cependant faire une jointure avec les 4 tables car il est demandé entre autres de sélectionner l'université (dans la table universites), la région (dans la table regions), le pays (dans la table pays) ou bien le rang (dans la table classements). Après avoir fait attention à la jointure, on met la condition de l'année avec un " ?" (comme elle est un paramètre de la fonction), on classe selon le rang avec "ORDER BY" et on se limite au 20 premières universités avec "LIMIT 20". Sur Python, on exécute en faisant bien attention à bien mettre annee sous forme d'un tuple et on renvoie une liste de tuples, comme demandé.

L'importation sur Excel prend exemple sur la fonction écrite précédemment et ne pose donc aucun problème particulier. De manière générale, aucune difficulté n'a été rencontrée lors de l'écriture de la fonction.

```python
curseur = connexion.cursor()
req = """
    SELECT c.rang, u.nom_univ, p.nom_pays, r.nom_reg, c.score_enseig, c.score_qualite_rech, c.score_the
    FROM classements as c
    JOIN universites as u
    ON c.id_univ = u.id_univ
    JOIN pays as p
    on p.id_pays = u.id_pays
    JOIN regions as r
    ON r.id_reg = p.id_reg
    WHERE c.annee = ?
    ORDER BY c.rang
    LIMIT 20
    """
curseur.execute(req, (annee,))
return [tuple(ligne) for ligne in curseur.fetchall()]
```

```python
top20 = top20_universites(connexion)
ws_top20 = wb.create_sheet("Top 20 Univ (Index)")
ws_top20.append(["Rang", "Université", "Pays", "Région", "Score Enseignement", "Score Recherche", "Score THE"])
for ligne in top20:
    ws_top20.append(ligne)
```

Fig. 10. Code de la fonction top20_universites (à gauche) et code menant à l'importation sur Excel (à droite)

3.4 stats_pib_vs_perf

Cette fonction compare les performances universitaires (à travers score_enseig, score_qualite_rech et score_the) selon le niveau de PIB par habitant.

Au départ, je pensais que le traitement devait se faire via Python car on doit ranger le PIB par habitant selon 3 classes : Faible, Intermédiaire et Élevé. Au départ j'avais donc pensé à seulement importer les données, non NULL car

sinon la comparaison selon les classes renverrait une erreur Python (on ne peut pas comparer un entier avec un None). On exécute ensuite la fonction en précisant l'année et j'ai décidé de créer 9 listes vides, servant à collecter les scores des universités pour chaque catégorie de PIB par habitant, et pour chaque type de score (enseignement, recherche, THE). Chacune de ces listes se remplit en parcourant les résultats de la requête SQL : pour chaque université, selon sa catégorie de PIB, ses scores sont ajoutés dans la liste correspondante. Une fois toutes les données collectées, on calcule la moyenne (grâce à une fonction moyenne() écrite au préalable) de chaque liste afin d'obtenir les performances moyennes par catégorie de PIB et par type de score. Enfin, ces moyennes sont regroupées dans un dictionnaire final, structuré par catégorie (Faible, Intermédiaire, Élevée) et par type de score (moy_ens, moy_rech, moy_the), ce qui permet de comparer facilement les performances universitaires selon le niveau de PIB par habitant.

Cependant, cette méthode est viable mais extrêmement longue et alourdit énormément le code. En relisant le TP de préparation, j'ai constaté l'existence de la clause CASE WHEN en SQL, qui permet de simplifier considérablement le code et de permettre de faire le traitement par SQL uniquement. On passe par ailleurs de 68 lignes de code à 35. Cela permet de regrouper directement les universités selon leur niveau de PIB et de calculer les moyennes dans la base de données, sans devoir créer de multiples listes en Python. Cette méthode rend le code plus compact, plus lisible et plus efficace, car tout est traité côté base de données. Par ailleurs, le fetchall() est remplacé par un fetchone() car on n'a plus qu'une seule ligne de résultats avec toutes les moyennes. On modifie aussi le dictionnaire : on le construit en utilisant les valeurs contenues dans la ligne. L'importation dans Excel se fait sans problème en reprenant le même modèle.

On notera donc qu'il y a des divergences entre les résultats obtenus et les résultats trouvés dans l'énoncé pour la catégorie "Faible". Cela s'explique par le fait que je ne prends pas en compte les valeurs NULL, car nous n'avons pas d'informations pour savoir dans quelle catégorie placer ces valeurs.

Au final, aucune réelle difficulté n'est à signaler pour l'écriture de la fonction, j'ai cherché à l'optimiser. La clause CASE WHEN est très intuitive.

Fig. 11. Schéma de transition : de la première version à la dernière version de stats_pib_vs_perf

3.5 stats_mixite_vs_perf

stats_mixite_vs_perf analyse le lien entre la mixité (ratio_fem) et les performances universitaires (à travers score_enseig, score_qualite_rech et score_the).

Cette fonction reprend le même modèle que stats_pib_vs_perf, suivant la même démarche algorithmique. Au départ, j'avais également fait une version non optimisée, de la même manière que stats_pib_vs_perf (cf Fig. 11). Après optimisation avec le traitement sur SQL avec la clause CASE WHEN, on obtient beaucoup moins de lignes. L'importation sur Excel se fait sans problème.

Comme pour la fonction précédente, aucune difficulté n'a été rencontrée, suivant la même logique algorithmique.

```python
curseur = connexion.cursor()
req = """
SELECT ratio_fem, score_enseig, score_qualite_rech FROM classements
WHERE annee = ?
AND ratio_fem IS NOT NULL
AND score_enseig IS NOT NULL
AND score_qualite_rech IS NOT NULL
"""
curseur.execute(req,(annee,))
sc_40_enseig = []
sc_40_60_enseig = []
sc_60_enseig = []

sc_40_rech = []
sc_40_60_rech = []
sc_60_rech = []

dict = {}
for ligne in curseur.fetchall():
    if ligne['ratio_fem'] < 40:
        sc_40_enseig.append(ligne['score_enseig'])
        sc_40_rech.append(ligne['score_qualite_rech'])
    elif 40 <= ligne['ratio_fem'] <= 60:
        sc_40_60_enseig.append(ligne['score_enseig'])
        sc_40_60_rech.append(ligne['score_qualite_rech'])
    elif ligne['ratio_fem'] > 60:
        sc_60_enseig.append(ligne['score_enseig'])
        sc_60_rech.append(ligne['score_qualite_rech'])

moyenne_40_enseig = moyenne(sc_40_enseig)
moyenne_40_60_enseig = moyenne(sc_40_60_enseig)
moyenne_60_enseig = moyenne(sc_60_enseig)

moyenne_40_rech = moyenne(sc_40_rech)
moyenne_40_60_rech = moyenne(sc_40_60_rech)
moyenne_60_rech = moyenne(sc_60_rech)

dict["<40%"] = {"moy_ens": moyenne_40_enseig, "moy_rech" : moyenne_40_rech}
dict["40-60%"] = {"moy_ens": moyenne_40_60_enseig, "moy_rech" : moyenne_40_60_rech}
dict[">60%"] = {"moy_ens": moyenne_60_enseig, "moy_rech" : moyenne_60_rech}
return dict
```

Transition →

```python
curseur = connexion.cursor()

req = """
SELECT
    AVG(CASE WHEN ratio_fem < 40 THEN score_enseig END) AS moy_ens_inf_40,
    AVG(CASE WHEN 40 <= ratio_fem AND ratio_fem <= 60 THEN score_enseig END) AS moy_ens_40_60,
    AVG(CASE WHEN ratio_fem > 60 THEN score_enseig END) AS moy_ens_sup_60,

    AVG(CASE WHEN ratio_fem < 40 THEN score_qualite_rech END) AS moy_rech_inf_40,
    AVG(CASE WHEN 40 <= ratio_fem AND ratio_fem <= 60 THEN score_qualite_rech END) AS moy_rech_40_60,
    AVG(CASE WHEN ratio_fem > 60 THEN score_qualite_rech END) AS moy_rech_sup_60
FROM classements
WHERE annee = ?
  AND ratio_fem IS NOT NULL
  AND score_enseig IS NOT NULL
  AND score_qualite_rech IS NOT NULL
"""

curseur.execute(req, (annee,))
ligne = curseur.fetchone()

return {"<40%": {"moy_ens": ligne["moy_ens_inf_40"],"moy_rech": ligne["moy_rech_inf_40"]},
        "40-60%": {"moy_ens": ligne["moy_ens_40_60"],"moy_rech": ligne["moy_rech_40_60"]},
        ">60%": {"moy_ens": ligne["moy_ens_sup_60"],"moy_rech": ligne["moy_rech_sup_60"]}}
```

Fig. 12. Schéma de transition : de la première version à la dernière version de stats_mixite_vs_perf

3.6 Compétences développées et pistes d'améliorations

J'ai pu à travers cette SAE être encore plus à l'aise avec les différents objets en Python (dictionnaires, tuples, listes, chaînes de caractères). J'ai pu développer mes compétences en optimisation de fonctions, à travers stats_mixite_vs_perf et stats_pib_vs_perf grâce à la clause CASE WHEN. J'ai également gagné en aisance dans l'utilisation conjointe de SQL et Python : avant cette SAE, les notions m'étaient encore floues et j'avais encore du mal à les appliquer. Aussi, j'ai réussi à apprendre réellement comment écrire des requêtes dynamiques grâce à la fonction rechercher_universites, ce qui m'a donc permis de comprendre encore mieux comment utiliser SQL et Python ensemble.

En termes de pistes d'améliorations, on pourrait proposer une analyse/interprétation des graphiques et/ou des résultats que nous obtenons. On pourrait aussi calculer différents indicateurs statistiques comme le V de Cramer, le rapport de corrélation, etc. pour observer la présence ou non d'une association. Aussi, on pourrait améliorer la qualité du pré-traitement des données, notamment sur la gestion des valeurs manquantes. Enfin, il serait intéressant de mettre en place une détection d'anomalies afin d'identifier des comportements atypiques ou des valeurs aberrantes.

Table des figures

1 ouvrir_connexion_bdd() (à gauche) et fermer_connexion_bdd() (à droite) 3
2 Code de la fonction evolution_universite 4
3 Code de la fonction top10_pays_multi_annees 4
4 Code de la fonction top5_pays_par_critere 6
5 Code de la fonction repartition_universites_par_region 7
6 Code de la fonction n_universites_par_indicateur 7
7 Code de la fonction stats_migration_vs_perf 8
8 Schéma de transition : de la première version à la dernière version de kpis_dashboard. 9
9 Code de la fonction rechercher_universites (à gauche) et resultat sur Excel en prenant comme paramètre nom_pays = "France", annee = "2025" et min_enseig = "40.0" (à droite) 10
10 Code de la fonction top20_universites (à gauche) et code menant à l'importation sur Excel (à droite) . 10
11 Schéma de transition : de la première version à la dernière version de stats_pib_vs_perf 11
12 Schéma de transition : de la première version à la dernière version de stats_mixite_vs_perf 12